

Learned Heuristics for Sound Algebraic Rewrite Search

Oracle-Derived Structural Potentials via Expert Iteration over Guarded Expression-Tree Transformations

Research Proposal — Internal Working Document v9

Ansuman Mullick • Bilkent University • ansuman.mullick@bilkent.edu.tr

May 2026 — Evolved v1–v8; v9 incorporates orchestrator architecture, mathematical ontology design, SIGMA positioning, and interpretability analysis

Abstract

Can oracle-derived structural potentials learn heuristics that beat hand-designed heuristics and strong classical search on held-out algebraic rewrite problems? We study this question in the setting of sound guarded rewrite search over symbolic algebra. The system casts equation-solving as search over typed expression-tree states with a validated library of ~ 57 guarded rewrite rules. Dense process supervision is derived automatically from distance to problem-specific canonical solution states (provided by symbolic solvers during training), providing executable soundness and per-step credit assignment with zero human labels. A three-phase design isolates contributions: (A) hand-designed structural heuristics + beam search, (B) supervised learned value function (benchmarking GIN, Tree-LSTM, and Transformer architectures) + beam search, and (C) expert iteration with learned policy + learned value + MCTS, where the policy is iteratively improved by training on its own best trajectories under oracle-derived reward. We address two foundational challenges explicitly: the value monotonicity problem (optimal paths often temporarily increase complexity) via non-myopic remaining-steps value training, and the parameterized action space (rewrite rules have argument families) via structurally-constrained instantiation yielding ~ 50 – 75 enumerable actions per state. Evaluation spans five axes: value fidelity against exact e-graph-extracted distances, final accuracy, equivalent-form invariance, candidate divergence detection, and invalid-step rate. Baselines include equality saturation extraction, A*/BFS, tool-using LLMs, and CoT LLMs. Strategy discovery is rigorously defined: solutions with strictly fewer rewrite steps than BFS-optimal. A calculus transfer pilot is included as an appendix experiment. The contribution is not a better CAS; it is a demonstration that executable structural potentials provide useful process supervision for sound rewrite search without manual step labels.

Beyond the single-agent algebra system, we present the architectural vision for an ontology-routed modular reasoning system: an LLM orchestrator directing domain-specific RL agents through a

mathematical ontology over transformation types, where each agent operates in a sound guarded action space and the CAS serves as training signal rather than inference dependency. GGMR is the first agent in this architecture; the orchestrator vision motivates but does not depend on the algebra results.

1. Motivation

1.1 The Credit Assignment Gap

ORMs reward entire chains on final-answer correctness. Math-Shepherd demonstrated automatic process supervision without human labels; ProcessBench showed earliest-error identification remains hard even for frontier models. Peano (Poesia & Goodman, 2023) showed that finite-valid-action symbolic math + RL is feasible; LEMMA extended this with learned abstractions/tactics for generalization. GGMR's novelty relative to this lineage is the combination of: (1) dense non-myopic structural potentials derived from canonical targets (not sparse completion reward), (2) executable soundness via explicit guard layer (not implicit in the action set), (3) rigorous evaluation against strong classical baselines including equality saturation, and (4) strategy discovery defined as shorter-than-BFS solutions.

1.2 The Squeeze Problem (Stated Honestly)

This paper operates in a squeeze between two failure modes, and we name them explicitly rather than pretending they don't exist:

If the rewrite library is too rich: classical search (BFS/A*) or equality saturation (e-graph extraction) will solve most problems efficiently, making the learned heuristic decorative. The learning contributes little beyond what a well-engineered CAS already provides.

If the rewrite library is too sparse: the policy cannot invent missing mathematical moves because the action space forbids them. Problems requiring techniques outside the library are unsolvable regardless of learning quality.

The experimental hypothesis is that a productive middle exists: problems where the rewrite library is complete enough to contain valid solution paths but the search space is large enough that naive enumeration is impractical, so learned heuristics provide genuine guidance. The paper's job is to demonstrate this middle exists, characterize its boundaries, and show that learned potentials outperform hand-designed heuristics within it. If the middle is empty or negligibly thin, the paper honestly reports a negative result.

1.3 Broader Context

LLMs score near-perfect on competition math benchmarks (AIME). At harder levels, structured systems (AlphaProof, AlphaGeometry2) outperform token-level reasoning. Recent hybrid systems (DeepSeek-Prover-V2, Aristotle) combine informal reasoning with formal proof search, making the landscape richer than a simple "token CoT vs. symbolic rewrites" dichotomy. GGMR does not claim to replace these systems; it investigates whether oracle-derived structural potentials are a useful training signal for sound rewrite search, complementary to existing approaches.

1.4 The Unified Architectural Principle

GGMR instantiates a pattern validated across an emerging research program: **hard structural constraints + soft learned signals**. LexCGraph (legal): relation whitelists + semantic similarity—structured nodes without edges perform worse than raw text. Fortunate Recall (memory): behavioral categories + decay/similarity—flat stores and cognitive categories lose to behavioral ontology. GGMR (math): guarded rewrites + learned structural potentials. **Unconstrained structuring is worse than no structuring; ontology-constrained structuring is better than both.**

This principle extends beyond individual systems to the orchestration layer itself (§9).

2. Two Foundational Challenges

2.1 The Value Monotonicity Problem

Problem: Optimal solution paths often temporarily increase structural complexity. Solving $(x^2 - 1)/(x - 1) = 3$ requires expanding and rearranging before collapsing to $x = 4$. Myopic distance-to-target penalizes productive steps.

Resolution (three layers):

Layer 1 — Prerequisite validation. Before coding: manually trace 20 problems through BFS-optimal paths. Measure monotonicity rate. If <60%, myopic distance is broken and the paper restructures around non-myopic value as the primary contribution (Phase A becomes a negative result demonstrating why naive structural distance fails).

Layer 2 — Non-myopic learned value. Phase B trains on (state, remaining-steps-to-target) pairs from solved trajectories, not pointwise distance. The network evaluates states by trajectory promise, seeing through complexity valleys. Analogous to a chess engine evaluating positions after a sacrifice.

Layer 3 — Search exploration. MCTS with UCB naturally explores branches that myopic heuristics reject. The policy learns complexification is sometimes productive because the non-myopic value function confirms it.

Consequence for error localization: We do NOT claim "first value increase = first error." Value trajectory analysis identifies candidate divergence regions—steps where the value signal deviates from the expected trajectory—not point-exact error locations. This is weaker than the v6/v7 claim but honest. ProcessBench confirms that exact earliest-error localization is hard even for frontier models; we target useful approximate localization, not perfection.

2.2 The Parameterized Action Space

Problem: Rewrite rules are parameterized (add what to both sides?). Unbounded parameterization makes the branching factor infinite.

Resolution: Parameterizations are constrained to structurally-motivated instantiations derived from the current expression tree: terms, coefficients, and factors already present as subexpressions. Effective branching factor: ~ 15 rule types $\times \sim 3\text{--}5$ structural parameterizations = $\sim 50\text{--}75$ enumerable actions per state. Policy outputs a distribution over (rule_id, target_subtree_index) pairs—a discrete, fully enumerable action space at each state.

2.3 The Abstraction Gap (Known Limitation)

GGMR's action space cannot invent auxiliary variables, novel substitutions, or lemmas not already expressible as compositions of primitive rewrites. This is a fundamental expressiveness limitation relative to systems like LEMMA (which induces learned abstractions) and hybrid informal/formal systems (which can generate arbitrary intermediate reasoning in natural language). GGMR partially addresses this through learned macro-actions—multi-step compositions discovered during expert iteration that effectively create higher-level actions. Full abstraction induction is future work.

3. Framework: Sound Guarded Rewrite Search

3.1 Expression-Tree Search

Equation-solving is modeled as search over typed expression-tree states. Each step applies a named, guarded rewrite rule with typed preconditions and postconditions. The goal is a problem-specific canonical end state (sets of equivalent forms, not single paths).

3.2 Rewrite Library (Honest Scale)

~ 20 axioms (field, order, equality), ~ 25 guarded rewrite rules (add/subtract/multiply/divide both sides, factor, expand, substitute, complete the square, quadratic formula, polynomial long division, partial fractions, rational root theorem, Vieta's, auxiliary variable substitution), ~ 12 guards/forbidden

operations (divide-by-zero, sqrt-negative, cancel-zero, square-without-check, domain violations). Total: ~57 rules. Coverage validated empirically against 500 reference solutions.

Soundness (precise claim): Every applied step is valid, assuming correct guard implementations and explicit side-condition tracking (domain assumptions, non-degeneracy conditions). Failures manifest as incompleteness (search exhaustion) or guard bugs, never unsound derivations. We do not claim completeness. "0% invalid-step rate" is a property of the rule generator, not a research result—it is reported for comparison against LLM CoT invalid-step rates, not as a contribution.

3.3 Relationship to Equality Saturation

Equality saturation via e-graphs (Willsey et al., POPL 2021, egg) compactly represents all equivalent forms and extracts optimal rewrites via graph extraction. This is a strong baseline, not a tangential reference. E-graph extraction may solve the "short" portion of the problem space efficiently; the research question is whether learned heuristics outperform extraction on longer/harder rewrite sequences where equality saturation hits scalability limits (Guided Equality Saturation explicitly documents these limits). E-graphs are used for: (a) computing gold-standard canonical targets, (b) computing exact shortest rewrite distances on tractable subsets, and (c) as a primary baseline (D2). Non-confluence in the rewrite system is the strategic landscape the policy learns to navigate.

3.4 Canonical Target Families

Targets are sets of equivalent canonical end states. Linear: $x = k$. Quadratic: factored zero-form or root set. Polynomial: fully factored or root enumeration. System: conjunction of assignments. To address oracle bias, we evaluate under multiple canonical target families (SymPy's canonical form, Wolfram Alpha's canonical form, mathematically-equivalent alternatives) to confirm the policy learns mathematical progress, not solver-specific path preferences.

3.5 Practical Interpretability: Auditable Execution, Opaque Policy

A critical architectural property: every step in the solution trace is a named, human-readable rewrite rule applied to a specific subtree with verified guard conditions. The full trace is auditable:

```
Step 1: SUBTRACT(5, both_sides) →  $x^2 + 2x = 3$ 
Step 2: COMPLETE_SQUARE(x) →  $(x+1)^2 = 4$ 
Step 3: SQRT(both_sides, ±) →  $x+1 = \pm 2$ 
Step 4: SUBTRACT(1, both_sides) →  $x = 1$  or  $x = -3$ 
```

Each step can be independently verified by a human or CAS. The **execution** is fully transparent. What remains opaque is the **policy**—the neural network deciding *which* rule to apply. The decision-making is a black box; the actions it produces are not.

This is a fundamental advantage over LLM chain-of-thought reasoning, where *both* the decision and the execution are opaque. In GGMR, the guard layer guarantees step validity regardless of why the step was chosen. The black box is shrunk from "everything" to "just strategy selection."

In the orchestrator architecture (§9), the ontology adds a second layer of interpretability: the routing decision (which agent handles which sub-problem) is determined by the ontology classification, not by opaque LLM judgment. The system can explain *which domain* was selected and *why* based on transformation type, even if the within-agent policy remains opaque.

4. Architecture: Three-Phase Design

4.1 Phase A: Hand-Designed Heuristic + Beam Search

No learning. Composite structural heuristic: tree depth ratio, variable isolation degree, operator/term counts. Beam search (B=5–10, depth 20). Known limitation: will fail on problems requiring intermediate complexification. Phase A's failure modes motivate Phase B. If prerequisite validation shows <60% monotonicity, Phase A is presented as a negative result.

4.2 Phase B: Learned Non-Myopic Value + Beam Search

Supervised learning on (state, remaining-steps-to-target) pairs from solved trajectories. Three architectures benchmarked: (a) Graph Isomorphism Network (GIN, 4–8 layers, ~2–5M params), (b) Tree-LSTM (recursive, natural for expression trees, avoids GIN oversmoothing), (c) Transformer over serialized prefix notation. Architecture selected by ordinal accuracy against exact e-graph distances.

4.3 Phase C: Expert Iteration with Learned Policy + MCTS

Terminology note: We use expert iteration (ExIt), not self-play. The system is a single agent solving problems with oracle-supplied targets—closer to oracle-guided policy improvement than competitive self-play (cf. STP, which uses actual conjecturer–prover competition). ExIt accurately describes the training loop: generate trajectories with current policy + MCTS, train on the best trajectories, iterate.

Policy network: Same architecture family as the value network. Takes current expression tree, outputs distribution over ~50–75 structurally-constrained (rule_id, target_subtree_index) actions.

Value network: Phase B's best architecture, fine-tuned during ExIt.

MCTS: Policy proposes top-N rewrites, value evaluates resulting trees, UCB drives exploration, distance-increasing branches deprioritized (not hard-pruned, to allow complexification paths).

Training loop: (1) Generate problem with known canonical target. (2) Policy + MCTS produces trajectories. (3) Per-step remaining-steps-to-target provides dense reward. (4) Policy updated toward

distance-reducing transforms; value updated toward better distance prediction. (5) Iterate over millions of problems. Zero human annotation.

4.4 What Expert Iteration May Discover

Strategic rewrite ordering: Problem-specific sequences more efficient than textbook order.

Intermediate complexification: Temporarily increasing complexity to enable cancellation—productive steps that hand-designed heuristics penalize.

Learned macro-actions: Multi-step compositions that effectively create higher-level actions not in the primitive rule set. These partially address the abstraction gap (§2.3).

Rigorous strategy discovery definition: A solution exhibits strategy discovery if it achieves strictly fewer rewrite steps than BFS-optimal through the rule space. BFS-optimal is computed via e-graph extraction on tractable subsets. Shorter-than-BFS is possible because ExIt + MCTS can learn compositional macro-actions that BFS treats as multi-step sequences. This is measurable, falsifiable, and not achievable by suboptimal alternative paths.

Honest expectation: If Phase C only modestly beats Phase B + beam/A*, the contribution is still valid (learned non-myopic value is the primary result) but the MCTS/ExIt framing should be demoted. We commit to reporting Phase C vs simpler search under matched compute budgets.

4.5 Value Robustness

Dense process rewards can cause training collapse (Gao et al., PRM Lessons). PROF demonstrates filter-based integration avoids collapse where direct reward shaping fails. Planned ablations: (a) value vs exact e-graph distance correlation, (b) filter-based (PROF-style) vs direct reward shaping, (c) policy entropy + value error over training iterations, (d) path-length controls, (e) oracle bias test: retrain with alternative canonical target families (SymPy vs Wolfram vs hand-written) and measure policy stability.

5. Experimental Plan

5.1 The Core Question

Can oracle-derived structural potentials learn heuristics that beat hand-designed heuristics and strong classical search (including equality saturation) on held-out algebraic rewrite problems?

5.2 Five Evaluation Axes

1. **Value fidelity (gold standard):** ordinal accuracy of hand-designed, supervised, and ExIt-refined value functions against exact shortest rewrite distances computed via e-graph extraction. Includes monotonicity rate measurement on BFS-optimal paths.
2. **Final accuracy:** problems solved within step limit under matched compute budget (node expansions or wall-clock time).
3. **Equivalent-form invariance:** policy/value stability under AC-equivalent and variable-renamed forms. If unstable, the system learned parser-dependent shape preferences, not mathematical progress. This is a central test, not a side property.
4. **Candidate divergence detection:** on synthetically corrupted traces with known ground-truth first wrong step, measure precision/recall of value-trajectory anomaly detection for identifying divergence regions (not point-exact errors).
5. **Invalid-step rate:** 0% by construction (generator property). Compared against LLM CoT rates. Reported for comparison, not claimed as contribution.

5.3 Ablation Grid

ID	Guards	Value	Policy	Search	Tests
A1	✓	None	Random	Random	Soundness floor
A2	✗	Hand (myopic)	Enum	Beam	Value w/o guards
A3	✓	Hand (myopic)	Enum	Beam	Phase A baseline
B1	✓	GIN (non-myopic)	Enum	Beam	Phase B (GIN)
B2	✓	Tree-LSTM (non-myopic)	Enum	Beam	Phase B (Tree-LSTM)
B3	✓	Transformer (non-myopic)	Enum	Beam	Phase B (Transformer)
C1	✓	Best-B (ExIt)	ExIt policy	MCTS	Phase C (full)
C2	✓	Best-B (ExIt)	ExIt policy	Beam	ExIt w/o MCTS
C3	✓	Best-B (ExIt)	ExIt policy	A*	ExIt w/ A* (matched)
C4	✗	Best-B (ExIt)	ExIt policy	MCTS	Full w/o guards
D1	✓	BFS/A* heuristic	BFS/A*	BFS/A*	Classical search
D2	✓	E-graph extraction	Eqsat	Eqsat	Equality saturation
E1	✗	(LLM)	(LLM)	CoT	Token-level LLM
E2	✗	(LLM)	(LLM)	Tools	Tool-using LLM

Key comparisons: A3 $\rightarrow$ B-best (learning beats hand heuristic?), B-best $\rightarrow$ C1 (ExIt beats supervised?), C1 $\rightarrow$ C2 $\rightarrow$ C3 (does MCTS justify itself vs beam/A* under matched budget?), C1 $\rightarrow$ C4 (guards matter?), C1 $\rightarrow$ D1 $\rightarrow$ D2 (learned heuristic beats classical search and eqsat?), C1 $\rightarrow$ E1 $\rightarrow$ E2 (structured search beats token-level?).

5.4 Generalization Splits

Split 1: Numerical. Same templates, new constants. Baseline.

Split 2: Template. Held-out equation structures (train on linear + quadratic separately; test on problems requiring both). Tests compositional rewrite transfer.

Split 3: Depth. Train ≤ 8 steps, test 12–20 steps.

Split 4: Strategy discovery. Count instances where C1 achieves strictly fewer steps than e-graph-extracted BFS-optimal. The acid test.

Split 5: Invariance stress test. AC-equivalent forms, variable renaming, operator reordering. Measure policy/value stability.

5.5 Oracle Bias Controls

Because training targets come from SymPy, the policy may learn solver-specific canonicalization preferences rather than general mathematical progress. Controls: (a) retrain with Wolfram Alpha-derived targets, measure policy divergence, (b) retrain with hand-written alternative canonical forms, (c) measure whether the policy performs differently on problems where SymPy and Wolfram choose different canonical forms. If the policy is oracle-invariant, it learned mathematics. If not, it learned SymPy.

5.6 Prerequisite Validation (Week 1)

Before any engineering: trace 20 problems (linear, quadratic, rational, polynomial) through BFS-optimal paths. Compute structural features at each step. Measure monotonicity rate. Outcome determines framing:

- $\geq 80\%$: Phase A is a meaningful baseline.
- 50–80%: Phase A is weak; paper emphasizes non-myopic value learning as primary contribution.
- $< 50\%$: Myopic distance is broken. Phase A is a negative result demonstrating why naive structural distance fails—itself an interesting finding.

5.7 Compute

~50–75 actions/state $\times$ ~15 depth. Policy/value: ~2–5M params. ExIt: 1–5M episodes on single GPU (B300 available), days. Problem generation: SymPy, unlimited, free. E-graph extraction: egg library on tractable subset (≤ 15 -step solutions). LLM baselines: API costs only.

5.8 Timeline (ICLR October 2026)

Phase	Work	When	Deliverable
0	Prerequisite: monotonicity test (20 problems)	Week 1	Go/pivot for Phase A framing
1	Algebra encoder + ~57 rules + e-graph distances + eqsat baseline	Jun–Jul	Expression encoder, rule library, D1/D2 baselines
2	Three value architectures (supervised) + Phase B	Jul–Aug	Architecture selection; A3 vs B1/B2/B3
3	Expert iteration + MCTS + Phase C	Aug–Sep	Full ablation C1–C4; strategy discovery
4	LLM baselines + oracle bias + invariance + robustness	Sep–Oct	E1/E2; all controls; paper writing
5	Calculus pilot (appendix) + submission	Oct	ICLR 2027 submission

6. Related Work (Comprehensive)

Symbolic math RL (closest prior work)

Peano (Poesia & Goodman, 2023): finite-action RL for algebra/arithmetic; showed primitive-action RL is feasible but plateaus on harder problems. **LEMMA**: adds learned abstractions/tactics on top of Peano; demonstrates abstraction induction is necessary for scaling. **ConPoLe**: contrastive policy learning for theorem proving. GGMR's delta: dense non-myopic structural potentials from canonical targets (Peano/LEMMA use sparse completion reward), explicit guard layer with side-condition tracking, rigorous comparison against eqsat and classical search baselines, and strategy discovery metric.

Equality saturation and term rewriting

egg (Willsey et al., POPL 2021): e-graph-based equality saturation. **Guided Equality Saturation**: intermediate guides for hard rewrite tasks; documents scalability limits. **Aurora**: RL interleaved with equality saturation for graph rewriting. Knuth-Bendix completion for confluence. GGMR uses eqsat as

baseline and gold-standard oracle, not competitor. The research question: where eqsat hits scalability limits, can learned heuristics extend reach?

Process reward models

Lightman (human labels), Math-Shepherd (automatic), ProcessBench (earliest-error hard), R-PRM, ThinkPRM. PRM Lessons (Gao et al.): naive PRM reward shaping causes training collapse. PROF: filter-based integration avoids collapse. GGMR: executable soundness + structural potentials + ExIt. Not a PRM—a structural value function for sound search.

Multi-agent mathematical reasoning (NEW in v9)

SIGMA (AAAI 2026): decomposes reasoning across four LLM-based specialist agents (Factual, Logical, Computational, Completeness) with a moderator synthesizer. Achieves 7.4% improvement on MATH500/AIME/GPQA. Critical distinction: SIGMA splits by *reasoning perspective* (what to think about); all agents are prompt-specialized LLMs with no soundness guarantees. GGMR's orchestrator vision (§9) splits by *execution modality* (how to do the work), using RL agents in guarded action spaces that are correct by construction. SIGMA's "Computational agent" reasons about computation in natural language; GGMR's RL agent executes computation through verified rewrite sequences. This is not an incremental difference—it is a different kind of system with different trust properties.

A-HMAD (2025): heterogeneous multi-agent debate with specialized roles. Same limitation: all agents are LLMs with different prompts, not structurally specialized executors.

Numina-Lean-Agent (2026): agentic formal reasoning with specialized tools (code execution, theorem search). Closer to GGMR's vision but Lean-specific and tool-integrated rather than ontology-routed.

Formal mathematics

AlphaProof (RL + Lean, Nature 2025), AlphaGeometry2 (neuro-symbolic geometry), DeepSeek-Prover-V2 (informal + formal hybrid), Aristotle (Lean + informal lemmas + geometry), HunyuanProver, InternLM2.5-StepProver, BFS-Prover (challenges MCTS necessity). GGMR targets accessible symbolic algebra, not formal proving. Different representation, different goal, complementary.

RL for multi-agent orchestration (NEW in v9)

A May 2026 survey on RL for LLM-based multi-agent systems identifies five orchestration sub-decisions (when to spawn, whom to delegate, how to communicate, how to aggregate, when to stop) and notes that explicit RL training for orchestration is still an open problem. GGMR's ontology-driven routing (§9) provides a structured prior for the "whom to delegate" decision, reducing it from an open-

ended LLM judgment to a deterministic classification—analogous to how FR's behavioral ontology reduces lifecycle management from LLM runtime decisions to deterministic policy.

Symbolic equation RL

Dabelow & Ueda (DQL + SymPy), O'Keeffe (curiosity-driven, graph actions), Sym-Q (offline RL for symbolic regression). GGMR adds: explicit guard layer, non-myopic value training, eqsat baseline, oracle bias controls, strategy discovery metric.

Novelty (precise)

Not "graphs + search + math" or "label-free process supervision." The specific combination: **dense non-myopic structural potentials from canonical targets, inside a sound guarded rewrite space with explicit side-conditions, evaluated against equality saturation and classical search baselines, with oracle bias controls and strategy discovery defined as shorter-than-BFS solutions.** No prior system combines this full set.

For the orchestrator vision (§9): no prior system combines **ontology-routed RL agents with sound guarded action spaces, orchestrated by an LLM, with CAS as training signal rather than inference dependency.** Every existing multi-agent math system uses LLM agents talking to LLM agents. The LLM-for-strategy + RL-for-execution separation does not exist in the literature.

7. Anticipated Reviewer Objections (Post-Adversarial Review)

"This is just term rewriting with a neural heuristic." Yes, architecturally. The contribution is demonstrating that oracle-derived structural potentials outperform hand-designed heuristics and classical search, with rigorous controls for oracle bias and invariance. That's an empirical ML question.

"Eqsat/egg solves this." Eqsat is baseline D2. The paper measures exactly where eqsat succeeds and where learned heuristics extend beyond it.

"Peano/LEMMA already did finite-action RL for algebra." Acknowledged as closest prior. GGMR adds dense non-myopic potentials, guard layer, eqsat baseline, strategy discovery metric.

"The value isn't monotone." Addressed in §2.1. Non-myopic training + MCTS exploration. Phase A failure is a feature, not a bug.

"LLMs already solve algebra." Correct. The question is whether learned structural potentials provide useful process supervision, not whether they beat GPT-5 on easy problems.

"MCTS may not justify itself." C1 vs C2 vs C3 under matched budgets. If MCTS doesn't win, we report honestly and demote it.

"This isn't self-play." Corrected in v8. It's expert iteration. Terminology updated throughout.

"0% invalid steps is trivial." Agreed. Reported for comparison, not claimed as contribution.

"The squeeze: too rich → eqsat wins; too sparse → unsolvable." Explicitly named in §1.2. The paper's job is to find and characterize the productive middle. If it's empty, we report that honestly.

"How is this different from SIGMA/multi-agent math?" (NEW in v9) SIGMA and related multi-agent systems use LLM agents with different prompts—all agents can hallucinate, none have soundness guarantees. GGMR's RL agents operate in guarded action spaces where every step is valid by construction. The trust model is fundamentally different. See §6 (Multi-agent mathematical reasoning) for detailed positioning.

"Why not just have LLMs call tools/CAS?" (NEW in v9) Tool calling makes the LLM the executor and the CAS the runtime dependency. GGMR uses CAS at training time to derive structural potentials; at inference, the RL agent runs independently with learned heuristics. Tool calls are stateless—the millionth call is no smarter than the first. The RL agent has learned intuition from millions of episodes. Tool calls produce answers; GGMR produces auditable solution traces. See §9.3 for the full architectural argument.

8. Future Vision

Cross-domain transfer

Compile rewrite modules for calculus, geometry, linear algebra. Measure whether ExIt-trained policies transfer across domains—transformation reuse, value function transfer, meta-strategy transfer. A calculus pilot is included in this paper's appendix.

Abstraction induction

Extending the action space with LEMMA-style learned tactics that go beyond primitive rewrite compositions. This addresses the fundamental expressiveness limitation (§2.3).

9. The Orchestrator Architecture (NEW in v9)

9.1 The Core Insight: Split Strategy from Execution

LLMs are extraordinary at strategic reasoning—knowing *that* you should complete the square before applying the quadratic formula, knowing *which* technique to try. What degrades their performance is sustained execution over long sequences: signs flip, terms drop, context degrades as the KV cache grows. The error rate compounds across steps.

This suggests a clean architectural split:

- **LLM orchestrator:** Strategy, routing, decomposition, web search, knowing *what* to do
- **RL agents:** Sound execution within guarded action spaces, knowing *how* to do it correctly

The orchestrator never executes a mathematical step. The RL agents never choose a strategy. Each operates in its domain of competence.

9.2 Mathematical Ontology over Transformation Types

The orchestrator routes sub-problems to agents via a mathematical ontology. Critically, this ontology classifies by **transformation type** (what operation is needed), not by **mathematical domain** (what topic the problem belongs to).

The domain-level ontology (Algebra, Geometry, Calculus, etc.) is the mathematical equivalent of FR's cognitive ontology (Semantic/Episodic/Procedural)—it groups by what things *are*, not by what should *happen* to them. A calculus integration problem might require algebraic simplification at step 3, a trigonometric identity at step 5, and a limit evaluation at step 7. Domain classification doesn't determine the action; transformation type does.

Candidate transformation-type categories (to be refined empirically, as FR's ontology was):

- **Structural simplification:** reduce, cancel, collect like terms
- **Strategic rearrangement:** isolate variables, change form, normalize
- **Decomposition:** factor, partial fractions, split into cases
- **Domain shift:** substitute, change variables, map to new representation space
- **Theorem application:** apply named results, invoke identities
- **Verification:** check solutions, validate domain constraints, confirm equivalence

The FR principle applies directly: **classify the step, not the problem**. Each sub-expression in the tree gets its own transformation-type classification. One problem, six steps, four different ontology categories. Each step routes to the agent whose action space contains that transformation type.

Empirical discovery process: Cluster thousands of worked solution steps (from textbooks, CAS solution traces, competition solutions) and find the natural groupings by transformation behavior—the same process that discovered FR's identity gravity well and motivated the 10+1 split. The ontology is discovered, not designed.

9.3 Why RL Agents, Not LLM Sub-Agents or Tool Calls

Against LLM sub-agents:

1. *Soundness.* LLM sub-agents can hallucinate, drop terms, make invalid cancellations. RL agents in guarded rewrite spaces cannot produce invalid steps—this is structural, not probabilistic.

2. *Verifiability*. Each RL step is a named rule with checked guards. Each LLM step is a token sequence requiring separate verification.
3. *Cost*. RL agent step: microseconds. LLM sub-agent step: full inference call.
4. *Consistency*. Same input → same output. No temperature sensitivity, no sampling variation.

Against tool calling (CAS at inference):

1. *Tool calls are answer-only*. SymPy.solve() returns $x = 4$ with no intermediate states, no process supervision, no learnable path.
2. *Formulation is the hard part*. The LLM must correctly formulate the CAS call—which function, which arguments, in which order. That formulation step is itself reasoning that can fail.
3. *No learning across problems*. The millionth CAS call is no smarter than the first. The RL agent develops heuristics—"when I see this structure, this rewrite sequence works"—from millions of training episodes.
4. *CAS as teacher, not executor*. GGMR uses CAS at training time to compute canonical targets and structural potentials. At inference, the RL agent runs independently with learned heuristics. This is the fundamental architectural distinction: the RL agent is what the CAS *taught*, running without the CAS.

The nuanced split: Not every sub-task needs RL. Recognition and pattern-matching ("this looks like integration by parts") is where LLMs excel. Execution ("perform the integration by parts correctly over 8 steps") is where they fail. The ontology determines not just *which* agent, but *what kind* of agent: LLM for recognition/strategy, RL for execution/transformation.

9.4 The Orchestrator's Toolkit

The orchestrator has three types of resources:

1. **RL agents** (sound execution): Domain-specific agents with guarded action spaces. Verified, auditable, deterministic. Used for mathematical transformation.
2. **Web search** (knowledge retrieval): Finding relevant theorems, identities, similar solved problems, known results. Unverified—enters as suggestions to the orchestrator, not as trusted steps.
3. **CAS tools** (spot verification): SymPy, Mathematica, egg for verifying intermediate results. Used for checking, not for solving.

9.5 The Verification Loop: Search, Confidence, Experiment

When the orchestrator encounters uncertainty, a three-tier verification process:

1. **High confidence** (multiple web sources agree, known identity): Orchestrator applies directly, RL agent executes.

- 2. **Medium confidence** (single source, plausible): An experiment agent tests the suggested technique on the current sub-problem within the guarded rewrite space. If the value function confirms improvement, commit. If not, backtrack.
- 3. **Low confidence** (conflicting sources, unusual claim): Discard or flag for alternative approach.

The experiment agent is a mathematician's scratch paper—cheap, sound, exploratory. It runs a few steps down a candidate path to assess viability without committing the full system. This implements the scientific method within the architecture: hypothesize (web search) → test (experiment agent) → confirm (CAS verification) → execute (domain agent).

9.6 Scaling: One Agent at a Time

GGMR is agent one (algebraic rewriting). The architecture scales by adding agents, not by redesigning the system:

- **Agent 1:** Algebraic rewriting (this paper)
- **Agent 2:** Calculus transformations (appendix pilot, then full paper)
- **Agent 3+:** Geometry, linear algebra, combinatorics, etc.

Each new agent requires its own rewrite library, guard set, and training pipeline. The ontology determines when a new agent is needed: when a cluster of transformation types emerges that existing agents can't handle, that's the signal to build a new agent. FR started as 8+1 categories; the identity gravity well revealed the need for 10+1. The mathematical ontology evolves the same way.

9.7 GGMR as Proof of Concept

The orchestrator vision motivates GGMR but does not depend on it. GGMR's job is to prove that a single RL agent in a guarded action space can learn useful heuristics for one mathematical domain. If that works, the orchestrator architecture becomes viable. If it doesn't, we know why and can characterize the limitations.

The paper presents GGMR as the algebra agent and the orchestrator as the architectural vision. The vision is described honestly as future work, with GGMR providing the first empirical test of the core mechanism.

10. Risk Assessment (Post-Review, Updated v9)

Risk	Severity	Mitigation
The squeeze: productive middle is empty	High	Characterize boundaries explicitly. If empty, report as negative result with analysis of why.

Risk	Severity	Mitigation
Eqsat baseline dominates	High	Test at increasing rewrite depth. Guided Equality Saturation documents scalability limits. Learned heuristics should win on longer sequences.
Non-myopic value doesn't improve over myopic	High	Prerequisite validation determines framing. Non-myopic vs myopic is an explicit ablation.
ExIt/MCTS doesn't beat supervised + beam/A*	Medium	Matched-budget comparison. If modest, demote ExIt; learned value is still primary contribution.
Oracle bias: policy learns SymPy, not math	Medium	Multi-target-family ablation. Cross-solver consistency test.
Peano/LEMMA comparison unfavorable	Medium	Different value signal (dense vs sparse), different baselines. Direct experimental comparison if reproducible.
Mathematical ontology categories wrong (NEW)	Medium	Empirical discovery via clustering worked solutions, same methodology as FR's ontology iteration. Start with candidate categories, refine via gravity well analysis.
Orchestrator vision too speculative for reviewers (NEW)	Low	Scope to appendix/future work. GGMR stands alone as algebra contribution. Orchestrator motivates but paper doesn't depend on it.

11. The Paper Claim

Within symbolic algebra, oracle-derived non-myopic structural potentials via expert iteration over a sound guarded rewrite system learn heuristics that outperform hand-designed heuristics, classical search, and equality saturation on held-out rewrite problems—providing executable process supervision and candidate divergence detection without human-labeled reasoning traces.

If the productive middle exists, this establishes oracle-derived structural potentials as a viable training signal for sound mathematical search. If it doesn't, the paper provides the first rigorous characterization of why, which is itself a contribution. Either outcome is publishable. The calculus transfer pilot and orchestrator vision are future work motivated by—but not dependent on—the algebra results.

The orchestrator architecture (§9) presents the broader vision: ontology-routed modular RL agents for mathematical reasoning, where LLMs provide strategy and RL agents provide sound execution. This is the next layer of a research program built on a validated architectural principle—**structured behavioral priors at ingestion prevent failures that no downstream inference can recover from**—applied across legal reasoning (LexCGraph), conversational memory (Fortunate Recall), and mathematical reasoning (GGMR).

v9 changes from v8: Added §3.5 (practical interpretability analysis); expanded §6 with SIGMA (AAAI 2026) positioning and multi-agent orchestration survey; added §7 new objections (SIGMA differentiation, tool-calling argument); added §9 (full orchestrator architecture: LLM orchestrator + ontology-routed RL agents + transformation-type ontology + verification loop + scaling strategy); moved unified architectural pattern from §9 to §1.4 and strengthened; updated risk assessment with ontology and reviewer-scope risks. Core v8 content (squeeze problem, three-phase design, ablation grid, timeline, prerequisite validation, related work) retained.